Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Anonymous[000]

Anonymous
anonymous@institute.com

Abstract. We mechanize two operational semantics for PROLOG with cut and prove their equivalence in the Rocq prover.
The first semantics is closely related to the one introduced by Debray and Mishra [3] for studying the optimizations employed by PROLOG abstract machines [20,17]. This semantics is based on a stack of alternatives, where the cut operator is represented by a pointer to a shorter stack.
The second semantics is based on And-Or trees, in which the structure of alternative computations is made explicit and preserved throughout execution. This enables an explicit characterization of the cut operator as a function that prunes selected branches of the tree. The tree-based semantics allows us to formulate introduction and elimination rules for disjunction in the presence of cut, which, as expected, differ from those of classical logic.
As a case study, we use the tree-based semantics to prove the soundness of the determinacy analysis introduced in [6]. By exploiting the equivalence between the two semantics, we relate this result to the stack-based semantics used in efficient implementations of logic programming such as [5].

1 Introduction

Backtracking is a compelling feature of logic programming, but it is also a double-edged sword. On the one hand, it makes it possible to write concise code that searches for a solution or enumerates all solutions. On the other hand, backtracking is often a source of poor performance and can make logic programs harder to debug. To mitigate these problems, the logic programming community has studied various forms of determinacy analysis [4,18,8,9,10,7,6] that identify logic programs that are deterministic, i.e., that compute functions rather than relations: they commit to their first solution and leave no *choice points* (places where backtracking could resume). A choice point corresponds to a suspended *alternative* computation, and in the following we use *choice point* and *alternative* as synonyms. The control operator *cut* allows the programmer to discard choice points and is therefore pervasively used to write deterministic code, but it is also well known to make the semantics of the programming language depart from a purely logical reading, thereby making reasoning about code harder.

Intuitively, nondeterminism arises when a computation can be completed using two different rules. This situation is typically ruled out statically by performing two complementary checks. The first one, called *mutual exclusion*, checks that at most one rule can apply to a given query. This condition is satisfied if either (i) no two rules have overlapping heads, so that no query can unify with both, or (ii) the analysis accounts for the presence of cut in rule premises, so that once the cut is executed all remaining rules are discarded. The second check, called *rule-body checking*, verifies that each rule either (i) calls only functions, ensuring that no choice point remains after the premises of the chosen rule have been executed, or (ii) executes a cut after calling relations, in which case any choice points left by the call are discarded.

Most papers in the literature only provide paper proofs for the soundness of their analysis, with the exception of Kriener and King [9]. They mechanize in the Rocq prover (formerly the Coq proof assistant) a denotational semantics for Prolog with cut and use it to prove the soundness of an abstract-interpretation-based determinacy inference algorithm. This mechanization has two shortcomings, in addition to the Rocq code being unavailable. First, the analysis does not scale to dynamic predicates (see [6, Sec. 8]), so it cannot be applied to λProlog [15,16]. Second, there is no mechanized link between their denotational semantics and the operational semantics of Debray and Mishra [3], which underlie standard Prolog abstract machines [20,1,17].

Contributions In this paper we mechanize in Rocq two operational semantics for Prolog with cut. The first is the stack-based semantics given by Debray and Mishra. This semantics is closely related to actual Prolog implementations, but it makes reasoning about the scope of cut difficult: alternatives are stored in a stack, in chronological order, and the cut operator is represented as a pointer to a shorter stack. The relation between the rules being cut and the affected stack frames is therefore not immediately visible.

To address this limitation, we introduce a semantics based on And-Or trees, in which the history of the computation is represented in a structured way: given a goal, each applicable rule generates an Or-branch, which is in turn followed by an And-branch for each premise. This enables an explicit characterization of the cut operator as a function that prunes selected branches of the tree. The tree-based semantics allows us to formulate introduction and elimination rules for disjunction in the presence of cut, which, as expected, differ from those of classical logic. For example, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut away by q_1.

Finally, we use the tree-based semantics to prove the correctness of the determinacy analysis from [6]. We prove in Rocq that any call to a predicate identified as deterministic leaves no additional Or-branches in the tree when it terminates. By composing this result with the equivalence between the two semantics, we obtain that any call to a deterministic predicate leaves no choice points on the stack used by actual Prolog implementations, including the Elpi [5] λProlog interpreter.

Our mechanization is axiom-free and is available at `ANONYMEZED`

2 Preliminaries: syntax and informal semantics of cut

In the entire paper we use Figure 1 as our running example, and we start by describing how we represent a program like that one. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ-calculus style (no parentheses).

The description of a program is given by a a pair $\mathbb{P} := (\text{list } \mathbb{R} \times \text{sigT})$ and is shared by both semantics. A program consists of a list of rules together with a signature environment sig. We delay the discussion of signatures to section 6 where we describe the static analysis that concerns them.

A rule is a pair $\mathbb{R} := (\mathbb{T} \times \text{list } \mathbb{A})$, where the first element represents its head and the second its list of premises. Each premise is an atom, i.e. either the cut operator or a term call. Terms include predicate symbols, data symbols, variables, and term application. The concrete definition of these two algebraic types is given below:

$$\mathbb{T} := \text{Symb } \mathbb{N} \mid \text{Pred } \mathbb{N} \mid \text{Var } \mathbb{N} \mid \text{App } \mathbb{T}\ \mathbb{T}$$
$$\mathbb{A} := \text{cut} \mid \text{call } \mathbb{T}$$

The syntax tree allows variables to be applied and predicates to be mixed with data. These higher-order features play no role in the current paper that focuses on first order logic programming, but allows future extensions to full λProlog to be based on the same mechanization.

Variables, predicates and data symbols are identified by natural numbers. We represent substitutions (of type Σ) as finitely supported maps from variables to terms, using the finmap library [14]. The empty substitution is written ε, while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics and it has the following type:

$$\text{backchain} : \mathbb{P} \to \mathcal{F}_\nu \to \mathbb{T} \to \Sigma \to (\mathcal{F}_\nu \times \text{list } (\Sigma \times \text{list } \mathbb{A}))$$

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_\nu$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

The function *unify* takes two terms t_1 and t_2, together with a substitution σ, as input, and optionally returns a substitution σ' that extends σ. Therefore, if t_1 and t_2 unify under σ, then $\sigma'\ t_1 = \sigma'\ t_2$, where $\sigma\ t$ denotes substitution application and $=$ denotes syntactic equality.

To keep the mechanization axiom-free, we implement a unification algorithm, which we briefly describe in section 6 and prove sound and complete. Since the two semantics and the static analysis use the same unification algorithm, the equivalence proof is completely agnostic to the particular implementation chosen for that algorithm.

```
func f int -> int.          % f is a deterministic predicate. The two rules
f 1 2.                      % have non-verlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int.          % r is a non-deterministic predicate. The heads
r 0 0.                      % overlap: the query for 0 has three applicable
r 0 1.                      % rules.
r 0 2 :- !.
func g int -> int.          % g is deterministic: if the first rule succeeds,
g A C :- r A B, f B C, !.   % the second one is cut away, as well as the
g D F :- f D E, f E F.      % choice points left by the call to r.
```

Fig. 1: Running example of a ELPI program

2.1 The cut operator

There are several variants of the cut operator in the literature. The most widely adopted one is the *hard cut* (see, e.g., the documentation of SWI-PROLOG [19]). Whenever the backchain function returns a non-empty list of alternatives, the first is explored immediately, while the others are suspended as choice points. Within a given alternative, premises are processed from left to right, executing each atom in turn. When a cut is encountered, it discards (i) all choice points created by backchaining for the current call and (ii) all choice points created while executing the premises to the left of the cut.

For example, consider the program in fig. 1 and the query `g 0 R`. Both rules for the predicate `g` apply to this query. Backchaining therefore returns the following two alternatives:

$$[(\sigma_1, [\texttt{r A B, f B C, !}]),\ (\sigma_2, [\texttt{f D E, f E F}])]$$

with $\sigma_1 \coloneqq \{\texttt{A} \mapsto \texttt{0}, \texttt{C} \mapsto \texttt{R}\}$ and $\sigma_2 \coloneqq \{\texttt{D} \mapsto \texttt{0}, \texttt{F} \mapsto \texttt{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_\nu$ any further.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1, i.e. `r 0 B`. The remaining alternatives are stored as choice points. Backchaining `r 0 B` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [\texttt{!}])]$ where $\sigma_3 \coloneqq \sigma_1 + \{\texttt{B} \mapsto \texttt{0}\}$; $\sigma_4 \coloneqq \sigma_1 + \{\texttt{B} \mapsto \texttt{1}\}$ and $\sigma_5 \coloneqq \sigma_1 + \{\texttt{B} \mapsto \texttt{2}\}$.

The next step of interpretation continues with `f B C` under σ_3, that is, `f 0 R`, and leaves $[(\sigma_4, []), (\sigma_5, [\texttt{!}])]$ as new choice points. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4, i.e. `f 1 R`. This succeeds with $\sigma_6 \coloneqq \sigma_4 + \{\texttt{R} \mapsto \texttt{2}\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for `r` and one from the second rule for `g` (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for `g` was selected. In contrast, a cut does not discard choice points created earlier; in this example, there are none. The final solution is: $\{\texttt{A} \mapsto \texttt{0}, \texttt{B} \mapsto \texttt{1}, \texttt{C} \mapsto \texttt{R}, \texttt{R} \mapsto \texttt{2}\}$.

In sections 3 and 4, we provide fully mechanized operational semantics for PROLOG with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

Notational conventions In the following section we note $\mathbb{B}$ for the boolean type. Following the Mathematical Components library, on which we depend, we use `x.1` and `x.2` to denote the first and second projection applied to the pair `x`. List concatenation is the infix ++, while list comprehension (i.e. mapping a function on a list) is written `[seq f x | x <- l]`.

3 And-Or Tree semantics

An And-Or tree is a stratified, variable-width tree, defined as follows:

$$\mathcal{T} := \text{KO} \mid \text{OK} \mid \text{Unexplored } \mathbb{A} \mid \text{Or (option } \mathcal{T}) \ \Sigma \ \mathcal{T} \mid \text{And } \mathcal{T} \text{ (list } \mathbb{A}) \ \mathcal{T}$$

It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer. These invariants about the $\mathcal{T}$ data type are precisely debscribed in section 5.1 where we introduce the valid_tree predicate.

The tree is built incrementally. The *Unexplored* node represents an unexplored branch (an atom). When the atom is a call, we use the backchaining procedure from section 2 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the *step* procedure described in section 3.2.

In addition to the Unexplored node we have two distinguished nodes, KO and OK, which represent respectively the smallest failed and successful tree.

The Or node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A. The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ, which becomes the current substitution when backtracking to B.

The And node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to the first choice point in A. We call B_0 the *reset point* for B: it represents the continuation for all alternatives of A except the first. That is, when backtracking occurs within A, the subtree B is replaced by the atoms in B_0. An example is shown in section 3.3.

A graphical representation of a tree is shown in fig. 3b. For compactness, we depict And and Or nodes as n-ary, with children associating to the right. The KO node acts as the terminating element of an Or list, while OK is the terminating element of an And list.

3.1 The semantics

The tree is constructed incrementally by two recursive functions, step and prune. Both traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved thanks to next_tree (in fig. 2). When this node is OK we say the entire tree is a *success*, when it is KO we say the tree *failed*, otherwise the tree is *incomplete*. In fig. 3 we mark with a black arrow the result of next_tree. The type of these two functions is given below.

$$\begin{aligned} \text{step} &: \mathbb{P} \to \mathcal{F}_\nu \to \Sigma \to \mathcal{T} \to (\mathcal{F}_\nu \times \text{tag} \times \mathcal{T}) \\ \text{prune} &: \mathbb{B} \to \mathcal{T} \to \text{option } \mathcal{T} \end{aligned}$$

The step routine, among its outputs, also returns a *tag*, whose definition is described in the very following section.

To avoid the constraint that all functions must terminate in Rocq, we define the transitive closure of step and prune as the inductive relation runT. More precisely runT iterates calls to step on incomplete trees to expand Unexplored nodes. When a tree fails it calls prune to find the next alternative and continues from there, if one exists.

```
Fixpoint next σ t : Σ * tree :=
  match t with
  | Unexplored _ | KO | OK ⇒ (σ, t)
  | Or None σ' B ⇒ next σ' B
  | Or (Some A) _ _ ⇒ next σ A
  | And A _ B ⇒ let (σ', pA) := next σ A in
                if pA == OK then next σ' B else (σ', pA)
  end.

Definition next_subst σ t := fst (next σ t).
Definition next_tree t := snd (next ε t).
Definition success t := next_tree t == OK.
Definition failed t := next_tree t == KO.
Definition incomplete t :=
  match next_tree t with Unexplored _ ⇒ true | _ ⇒ false end.
```

Fig. 2: next routine and derived functions

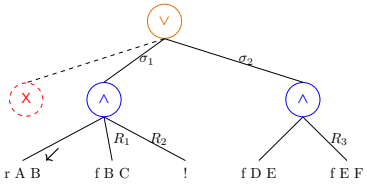
3.2 The step procedure

The step procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a tag, and an updated tree.

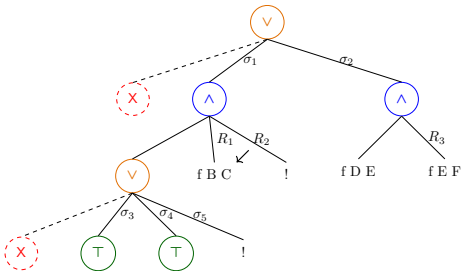
The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

g 0 R

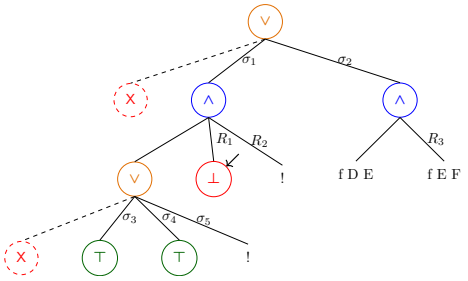
(a) Initial query



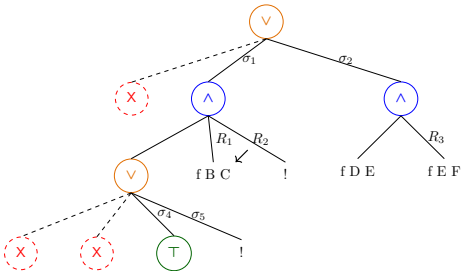
(b) Tree after step on `g 0 R`



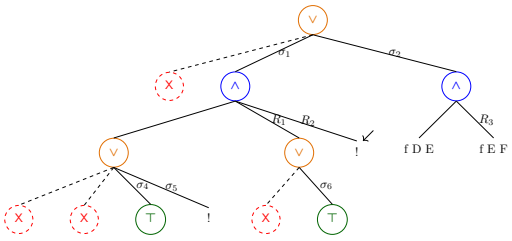
(c) step on fig. 3b



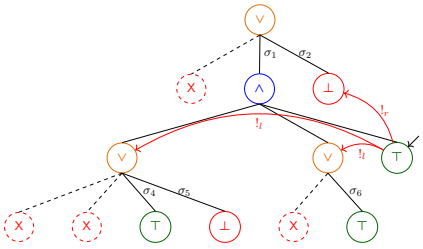
(d.1) step on fig. 3c



(d.2) prune on fig. 3d.1



(e) step on fig. 3d.2



(f) step on fig. 3e

Fig. 3: Execution in the tree semantics. The substitutions $\sigma_1 \dots \sigma_6$ are from section 2.1. R_1, R_2, and R_3 are reset points and correspond respectively to the lists of atoms [`f Y Z, !`], [`!`], and [`f Y Z`].

The tag is implemented as follows:

$$tag := \text{Expanded} \mid \text{CutBrothers} \mid \text{Failed} \mid \text{Success}$$

It indicates which kind of step was performed. Failure and Success are returned for trees that satisfy, respectively, the failed and success tests. CutBrothers indicates the interpretation of a superficial cut: step was called on an And-layer and its next_tree is the cut atom. Finally, Expanded accounts for the interpretation of predicate calls and non-superficial cuts.

The step procedure evaluates atoms. In particular, it leaves the tree unchanged when the tree is *success* or *failed*.

Lemma 1 (succ_step_iff). $\forall\ p\ v\ \sigma\ t,\ \text{success}\ t \leftrightarrow \text{step}\ p\ v\ \sigma\ t = (v,\ \text{Success},\ t)$

Lemma 2 (fail_step_iff). $\forall\ p\ v\ \sigma\ t,\ \text{failed}\ t \leftrightarrow \text{step}\ p\ v\ \sigma\ t = (v,\ \text{Failed},\ t)$

The step procedure expands Unexplored nodes by calling backchain, which returns a list l of alternatives. If l is empty, then the current call atom is replaced by KO. Otherwise, it is replaced by a right-skewed tree of Or nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is OK; otherwise, it is a right-skewed tree of And nodes.

Figure 3 depicts the tree corresponding to the running example (section 2.1) as it undergoes calls to step and prune. We run query `q 0 R` against the program P in fig. 1. Let c_0, c_1, and c_2 be the children of the root. By construction, c_0 is None, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P. Note that c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in section 2.1. Reset points are defined as follows: $R_1 := [\texttt{f Y Z, !}]$, $R_2 := [\texttt{!}]$, and $R_3 := \texttt{f Y Z}$. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of step performing a tree expansion via backchain appear in figs. 3c to 3e.

The interpretation of a cut The step corresponding to a cut performs two actions: (i) the cut node is replaced by OK; and (ii) the subtrees in the scope of Cut are pruned. More precisely, (ii) prunes the left siblings of the Cut node (in the same And-layer, denoted $!_l$) and prunes the right uncles of Cut (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in fig. 3f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. Note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by KO. This amounts to discarding the third rule of predicate `r`.

3.3 The prune procedure

When backchain returns an empty list, step produces a tree that *failed* and the computation must backtrack. The prune procedure is responsible for producing the new tree from which the computation can continue.

In fig. 3d.1 we encounter a failure because no rule applies to the atom `f B C` under substitution σ_3, which maps `B` to `0`. The prune procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the OK node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the KO node with the information stored in the reset point R_1. This restores the call `f B C`, which can then be reconsidered under substitution σ_4.

Furthermore, prune serves another important purpose. Its behavior depends on the boolean flag it receives as input: prune false t recursively removes all failures (if any) from t, whereas prune true t removes the first success in t. For example, the tree in fig. 3f contains a successful path that maps `R` to `2`. Calling prune on this tree returns None, since there are no alternatives in the tree after the success.

Some interesting properties of prune are shown below; they illustrate how prune complements step with respect to failed, successful, and incomplete trees.

Lemma 3 (incpl_prune). $\forall\ b\ t,\ \text{incomplete}\ t \to \text{prune}\ b\ t = \text{Some}\ t$

Lemma 4 (prune_Some). $\forall\ b\ t\ t',\ \text{prune}\ b\ t = \text{Some}\ t' \to \text{failed}\ t' = \text{false}$

Lemma 5 (prune_None). $\forall\ t,\ \text{prune false}\ t = \text{None} \to \text{failed}\ t = \text{true}$

3.4 The runT relation and its properties

The inductive relation runT has the following type:

$$\text{runT} : \mathbb{P} \to \boldsymbol{\mathcal{F}_\nu} \to \Sigma \to \mathbb{T} \to \text{sol} \to \mathbb{B} \to \boldsymbol{\mathcal{F}_\nu} \to \texttt{Prop}$$

It associates a program, a set of variables, an initial substitution σ, and a tree t with a solution r, a boolean b, and an updated set of variable names v'.

The type of sol is defined as follows:

$$\text{sol} := \text{Zero} \mid \text{One}\ \Sigma \mid \text{Many}\ \Sigma\ \mathcal{T}$$

Zero represents a failing execution. One σ represents an interpretation producing exactly one solution, i.e. no backtracking is possible from the given state. It also carries the substitution σ that makes the interpretation succeed. Finally, Many σ t represents an execution in which the original state produces the solution σ, while t represents the remaining unexplored alternatives.

The boolean b records whether a superficial cut was evaluated during execution, and v' is the updated set of variables tracking freshly generated names.

The runT predicate has five cases, depicted as inference rules in fig. 4.

Rules StopOT and StopMT say that if the tree t is success, then the substitution σ' is extracted from t (starting from σ), depending on the result of prune true t – aiming to clean the tree from its successful path – we return One or Many, if there exist alternatives to be explored. Rule StepT applies when next_tree t is an atom, then step is invoked to evaluate t, and runT is called recursively on the updated tree. When next_tree t is a failure, rule BackT calls prune to clear the failed path; if the resulting cleaned tree exists, runT is called recursively on it. Finally, rule FailT accounts for trees with no solutions.

$$\frac{\text{success } t \qquad \text{next\_subst } \sigma\ t = \sigma' \qquad \text{prune true } t = \text{None}}{\text{runT } v\ \sigma\ t\ (\text{One } \sigma')\ \text{false } v}\ \text{StopOT}$$

$$\frac{\text{success } t \qquad \text{next\_subst } \sigma\ t = \sigma' \qquad \text{prune true } t = \text{Some } t'}{\text{runT } v\ \sigma\ t\ (\text{Many } \sigma'\ t')\ \text{false } v}\ \text{StopMT}$$

$$\frac{\text{incomplete } t \qquad \text{step } p\ v\ \sigma\ t = (v',\ \text{tg},\ t') \qquad \text{runT } v'\ \sigma\ t'\ r\ b\ v''}{\text{runT } v\ \sigma\ t\ r\ ((\text{tg}\ =_b \text{CutBrothers})\ ||\ b)\ v''}\ \text{StepT}$$

$$\frac{\text{failed } t \qquad \text{prune false } t = \text{Some } t' \qquad \text{runT } v\ \sigma\ t'\ r\ n\ v'}{\text{runT } v\ \sigma\ t\ r\ n\ v'}\ \text{BackT}$$

$$\frac{\text{prune false } t = \text{None}}{\text{runT } v\ \sigma\ t\ \text{Zero false } v}\ \text{FailT}$$

Fig. 4: Rule system for runT

Lemma 6 (runT_det).

$$\forall\ p\ v_0\ \sigma\ t\ r\ r'\ b\ b'\ v\ v',\ \text{runT } p\ v_0\ \sigma\ t\ r\ b\ v \rightarrow \text{runT } p\ v_0\ \sigma\ t\ r'\ b'\ v' \rightarrow r' = r \wedge v' = v \wedge b' = b$$

The proof is by induction on the first runT derivation and is immediate, because the rules are selected based on next_tree t. In particular, the hypotheses success t, incomplete t, and failed t are mutually exclusive. Moreover, by lemma 5, the hypothesis of FailT implies that t *failed*; hence FailT is exclusive with StopOT, StopMT and StepT, and it is also exclusive with BackT since they impose different requirements on the output of prune.

The boolean output of runT allows us to state interesting properties about the Or node in the presence of cut. Below, we report only a selection of the lemmas that were proved. To simplify their presentation, we introduce the following notation for Or nodes. When the left-hand side is None, we write the symbol $\square$. Otherwise, the notation $A \vee B_\sigma$ implicitly denotes (Some A) $\vee\ B_\sigma$.

Lemma 7 (or_intro_left).

$$\forall\ p\ v\ v'\ \sigma\ \sigma'\ A\ A'\ \sigma_1\ B,\ \text{runT } p\ v\ \sigma\ A\ (\text{Many } \sigma'\ A')\ \text{true } v' \rightarrow \text{runT } p\ v\ \sigma\ (A \vee B_{\sigma_1})\ (\text{Many } \sigma'\ (A' \vee \text{KO}_{\sigma_1}))\ \text{false } v'$$

Lemma 8 (or_intro_right).

$$\forall\ p\ v\ v'\ \sigma\ A\ \sigma_1\ B,\ \text{runT } p\ v\ \sigma\ A\ \text{Zero true } v' \rightarrow \text{runT } p\ v\ \sigma\ (A \vee B_{\sigma_1})\ \text{Zero false } v'$$

Lemma 9 (or_intro_right2).

$$\forall\ p\ v_0\ v_1\ v_2\ \sigma\ A\ \sigma_1\ \sigma_2\ B\ b,$$
$$\text{runT}\ p\ v_0\ \sigma\ A\ \text{Zero false}\ v_1 \to \text{runT}\ p\ v_1\ \sigma_1\ B\ (\text{One}\ \sigma_2)\ b\ v_2 \to$$
$$\text{runT}\ p\ v_0\ \sigma\ (A \vee B_{\sigma_1})\ (\text{One}\ \sigma_2)\ \text{false}\ v_2$$

Lemmas 7 to 9 correspond to the introduction rules for disjunction. If A executes a cut, one *cannot* obtain $A \vee B$ from B: the execution of A, whether it ultimately succeeds or fails, makes the success of B irrelevant (the cut discards it). In the first lemma, B is forced to be KO; in the second one, the failure of A absorbs B. The last lemma is connected to the depth-first exploration of the tree: proving a disjunction from its right-hand side can only occur after the left-hand side has been disproved. Depicted as rules:

$$\frac{A \quad (A \text{ cuts})}{A \vee B}\ (\vee_{il}) \qquad \frac{\neg A \quad (A \text{ cuts})}{\neg(A \vee B)}\ (\vee_{ir1})$$

$$\frac{\neg A \qquad B \quad (A \text{ does not cut})}{A \vee B}\ (\vee_{ir2})$$

Lemmas 10 and 11 are elimination rules for disjunction.

Lemma 10 (or_elim1).

$$\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ A'\ B\ B',$$
$$\text{runT}\ p\ v\ \sigma\ (A \vee B_{\sigma_1})\ (\text{Many}\ \sigma'\ (A' \vee B'_{\sigma_1}))\ \text{false}\ v' \to$$
$$\exists\ b,\ \text{runT}\ p\ v\ \sigma\ A\ (\text{Many}\ \sigma'\ A')\ b\ v' \wedge B' = \text{if}\ b\ \text{then KO else}\ B$$

Lemma 11 (or_elim2).

$$\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ B\ B',$$
$$\text{runT}\ p\ v\ \sigma\ (A \vee B_{\sigma_1})\ (\text{Many}\ \sigma'\ (\square\ \vee B'_{\sigma_1}))\ \text{false}\ v' \to$$
$$(\exists\ b,\ \text{runT}\ p\ v\ \sigma\ A\ (\text{One}\ \sigma')\ b\ v' \wedge (b = \text{true} \to B' = \text{KO})) \vee$$
$$(\exists\ v_2\ b,\ \text{runT}\ p\ v\ \sigma\ A\ \text{Zero false}\ v_2 \wedge \text{runT}\ p\ v_2\ \sigma_1\ B\ (\text{Many}\ \sigma'\ B')\ b\ v')$$

From $A \vee B$, when A is not inert (i.e. not already explored), we cannot deduce A or B independently. Instead, we can only derive a weakened conclusion of the form $\top \vee B'$, where B' provides no information in the case where the exploration of A performs a cut. This yields an elimination rule that is stronger than the usual one. If A does not cut, the elimination rule has the usual two premises, together with the additional constraint that whenever B holds, A must have failed. This again reflects the depth-first traversal of the proof tree.

$$\frac{A \vee B \qquad \overset{[A]}{C} \qquad (A \text{ cuts})}{C}\ (\vee_{e1})$$

$$\dfrac{A \vee B \qquad \begin{matrix}[\mathrm{A}] \\ C\end{matrix} \qquad \begin{matrix}[\neg A, B] \\ C\end{matrix} \quad (A \text{ does not cut})}{\mathrm{C}}\ (\vee_{e2})$$

It is worth recalling that the "A cuts" side-condition in the rules above is precisely the boolean result returned by runT. Without this information, it is impossible to formulate these rules: it is not merely a matter of whether A contains a syntactic occurrence of cut, but whether that cut is actually executed during the (dis)proof of A (in rules $(\vee_{ir1})$ and $(\vee_{ir2})$). Indeed, an atom preceding the cut may fail, so the cut is never reached.

As anticipated in the introduction, these lemmas contradict the commutativity of disjunction.

4 Stack-based semantics

The stack-based semantics we present is very close to what is implemented by the Elpi interpreter. This semantics is similar to the one proposed by Pusch in [17], which is in turn closely related to the semantics given by Debray and Mishra in [3, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [20,1].

The execution state is a stack of alternatives. Each alternative is a list of goals. Intuitively it amounts to a disjunctive normal form: each stack item is a self contained computation, coupling a substitution with all the goals to be solved.

Goals pair an atom with a list of the *cut-to* alternatives, making the two datatypes mutually recursive, but these alternatives have a very different role. They have to be understood as pointers to lower levels of the stack itself. This datum is used to implement the cut, i.e. to prune the stack of alternatives.

```
Inductive alts  := nilA | consA of (Σ * goals)  & alts
with       goals := nilG | consG of (Atom * alts) & goals
```

The stack-based semantics is described by the runS inductive predicates.

$$\mathrm{runS} : \mathcal{F}_{\mathcal{V}} \to \texttt{alts} \to \mathrm{option}\ (\Sigma \times \texttt{alts}) \to \texttt{Prop}$$

The runS predicates relates a set of variables and a list of alternatives to optional result. None stands for failure, i.e., no solution, whereas Some (σ, a) represent success with σ begin the solution and a as the list of non-yet-explored alternatives.

The rule system for runS is given in fig. 5 and is made by 4 cases. We distinguish two main cases. If we run out of alternatives we fail: rule FailS stops the execution and return None. If the list of alternatives is $(\sigma_0, h) :: a$, then we look at the list of goals h (under the substitution σ_0). If h is empty, rule StopS stops the execution with a success σ_0 leaving a as the unexplored alternatives.

$$\mathcal{B}_S\ p\ v\ t\ \sigma\ a\ g = \left[\sigma', ([(p, q, a) \mid q \in b] +\!\!+\ g) \;\middle|\; (\sigma', b) \in bc(p, v, t, \sigma)\right].$$

$$\frac{}{\text{runS } v\ ((\sigma,\ [])\ ::\ a)\ (\text{Some } (\sigma,\ a))}\ \text{StopS}$$

$$\frac{\text{runS } v\ ((\sigma,\ g)\ ::\ \text{ca})\ r}{\text{runS } v\ ((\sigma,\ (\text{cut},\ \text{ca})\ ::\ g)\ ::\ \_)\ r}\ \text{CutS}$$

$$\frac{\mathcal{B}_S\ p\ v\ t\ \sigma\ a\ g = (v',\ \text{bs}) \qquad \text{runS } v'\ (\text{bs} +\!\!+\ a)\ r}{\text{runS } v\ ((\sigma,\ (\text{call } t,\ \_)\ ::\ g)\ ::\ a)\ r}\ \text{CallS}$$

$$\frac{}{\text{runS } \_\ []\ \text{None}}\ \text{FailS}$$

Fig. 5: Rule system for runS

If h is not empty we look at its head goal (t, ca) where t is the atom and ca is its cut-to alternatives. If t is a cut, rule CutS continues to evaluate the tail of h under with alternatives ca. Finally, when t is a call, rule CallS backchains: for each rule it pairs each premise with the current stack of alternatives, and prepends the obtained goals to the tail of h. Note that if backchain returns the empty list, the second premise of rule CallS amounts to exploring the next item in the current stack.

4.1 Example of execution

We propose an example of the execution of runS in fig. 6. The arrows in the images represent the cut-to pointers.

We have not drawn arrows coming out of call atoms since the interpreter ignores the cut-alternatives in this case. The evolution of the stack in fig. 6 mirrors the example in section 2.1: we evaluate the first goal of the first alternative. During backchaining, we add a pointer to the remaining alternatives to each new cut operator.

If a cut is evaluated, we replace the remaining alternatives with the cut-to alternatives. For example, in fig. 6f, evaluating the cut discards the second and third elements in the stack. Backtracking is performed by simply consuming the first alternative from the stack, as shown in fig. 6d.

4.2 Inadequacy of the stack-based semantics for formal reasoning

The lemmas that we presented in section 3.4 are hard to express in the stack-based semantics, due to the lack of structure in the stack to delimit the scope of the cut operator. For example, consider a stack $a_0, a_1, \ldots, a_n$. If a_0 succeeds but executes a cut, it is hard to relate the remaining alternatives $a_1, \ldots, a_n$ to

the alternatives $b_1, \ldots, b_m$ that survive the cut, since the only simple invariant is that $b_1, \ldots, b_m$ is a suffix of $a_1, \ldots, a_n$. If $a_1, \ldots, a_n$ were generated before the call whose execution generates the cut in a_0, then they all stay; but if some were generated afterwards, they are discarded. Expressing this precisely would require attaching, at the very least, time-stamps (or depths) to goals, which amounts to a cumbersome encoding of a tree.

5 Equivalence of the two semantics

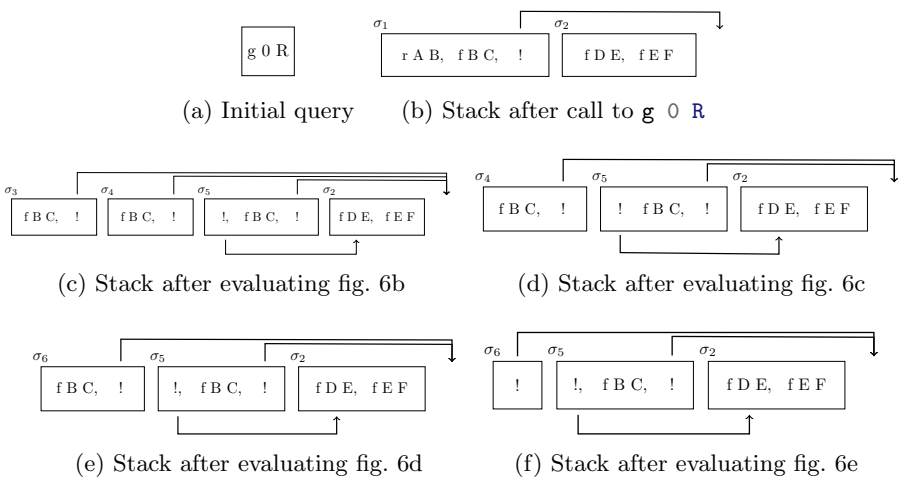
In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. We define a translation from trees to stacks, called tree_to_stack, but we do not explicitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [2].

Before describing the translation of trees to stacks (in section 5.2) we need to define the notion of *valid tree*. The tree datatype indeed contains trees that cannot be generated by runT via step or prune, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

5.1 Valid trees

The class of valid trees is captured by the valid_tree function shown in Figure 7. While all base cases of tree are valid, we need to analyze the Or and And constructors more carefully.

For the Or constructor, we distinguish two cases depending on whether the left-hand side is present. If it is not, then the right-hand side must be a valid



(a) Initial query (b) Stack after call to `g 0 R`

(c) Stack after evaluating fig. 6b (d) Stack after evaluating fig. 6c

(e) Stack after evaluating fig. 6d (f) Stack after evaluating fig. 6e

Fig. 6: Example of execution

tree. Otherwise, the left-hand side must be a valid tree, and the right-hand side must either be the KO tree or be unexplored. The former case occurs when the right-hand side is cut away by the evaluation of a superficial cut in the left-hand side. In the latter case we use the *base_or* predicate to check that B is the right-skewed tree formed by a disjunction of conjunctions, generated after a successful backchain for a call.

For the And constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. B must be the right-skewed tree containing conjunctions of premise atoms. This is enforced using the *big_and* procedure, which generates a tree from the reset point B_0 (the same procedure used to transform each alternative output by backchain into the And-layer of the resulting tree). When the left hand side is successful, then the right hand side may have been explored, and consequently must be a valid tree.

We point out the two main invariants that stating that valid trees are preserved by calls to step and prune.

Lemma 12 (valid_tree_step).

$$\forall\ \sigma\ v\ A\ r,\ \text{valid\_tree}\ A \to \text{step}\ u\ p\ v\ \sigma\ A = r \to \text{valid\_tree}\ r.2$$

Lemma 13 (valid_tree_prune).

$$\forall\ A\ B\ b,\ \text{valid\_tree}\ A \to \text{prune}\ b\ A = \text{Some}\ B \to \text{valid\_tree}\ B$$

We can therefore derive how valid trees are preserved by the stack interpreter.

Theorem 1 (valid_tree_run). $\forall\ b\ \sigma\ \sigma'\ v\ v'\ A\ B$,

$$\text{valid\_tree}\ A \to \text{runT}\ u\ p\ v\ \sigma\ A\ (\text{Many}\ \sigma'\ B)\ b\ v' \to \text{valid\_tree}\ B$$

```
Fixpoint valid_tree A :=
  match A with
  | Unexplored _ | OK | KO ⇒ true
  | Or None _ B ⇒ valid_tree B
  | Or (Some A) _ B ⇒ valid_tree A && ((B == KO) || base_or B)
  | And A B0 B ⇒ valid_tree A && if success A then valid_tree B
                                 else B == big_and B0
  end.
```

Fig. 7: Valid tree

5.2 From trees to stacks

The tree_to_stack recursive function translates a tree to a stack.

$$\text{tree\_to\_stack} : \mathcal{T} \to \Sigma \to \text{alts} \to \text{alts}$$

The function tree_to_stack takes as input the tree t_0, a substitution σ for the input tree, and an auxiliary list of choice points *cp*. These choice points represent alternatives that appear at the same level of the current tree, such as, for example, the right siblings of an Or node.

The OK node represents one successful alternative; therefore it is translated into a singleton list whose only element has an empty list of goals. The KO node represents failure, hence it is mapped to the empty list of alternatives. Finally, atoms are mapped to a singleton list containing one alternative whose only goal is that atom, with an empty cut-to list. At this stage, the cut-to list cannot point to *cp*: atoms are not right-uncle subtrees, whereas *cp* represents alternatives that will actually be discarded if the atom turns out to be a cut. The correct cut-to list is therefore determined later, once the trees corresponding to sibling subtrees have been inspected.

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B. The alternatives of B are valid choice points for A and should be passed to the translation of A. The two lists of alternatives can then be concatenated. Moreover, every atom occurring one level below *cp* (i.e. in A or B) should add *cp* as its cut-to alternatives.

The translation of $A \wedge_{B_0} B$ starts with the translation of A. If the translation of A is an empty list we return the empty list of alternatives without even touching B nor B_0, since an empty translation for A indicates failure and this in turn implies the failure of the entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$, the B node represents the continuation of the first alternative g, whereas B_0 is the continuation for each alternative in a.

Example of tree_to_stack execution The translations of the trees in fig. 3 are shown in fig. 6. The letters in the figures relate each tree to the corresponding stack. For example, 3b is translated into 6b. We also note that 3d.1 and 3d.2 are both translated into 6d, showing that tree_to_stack is not injective: different trees can map to the same stack. Consequently, there are transitions in runT that are not visible in runS. As an example of a call to tree_to_stack, we take the tree and the stack in fig. 8, which are respectively taken from figs. 3c and 6c. The red arrows point from the head of an alternative in the tree to the head of the corresponding cell in the stack. We focus on the deepest Or node in the tree. This subtree is a disjunction with four children. The first is ignored, since it is None. The success node below σ_3 has `f B C` and the cut operator as continuation, corresponding to the first cell in the stack. We note that the cut operator points outside the stack, since the scope of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is the list of goals `f B C, !`; this is translated into the second cell of the stack. Finally, the

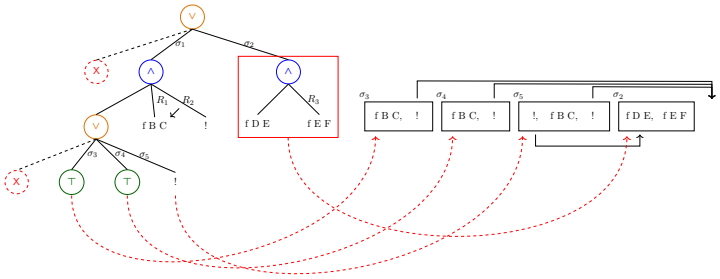


Fig. 8: Example of tree_to_stack execution

cut operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative. We can see that the first cut points to the translation of the subtree under σ_2, since it is one Or-layer above its right uncles.

5.3 Equivalence proof

The tree-based semantics exposes more information than needed, hence we hide it behind existentials.

Definition 1 (runT'). $\lambda\ p\ v\ \sigma\ t\ r.\ \exists\ b\ v',\ \text{runT}\ p\ v\ \sigma\ t\ r\ b\ v'$

From figs. 3 and 6, we observe that, upon backtracking, runT may perform more steps than runS before returning a solution or a failure. These silent transitions must be skipped when proving the equivalence between the two semantics. Therefore, we prove the following lemma.

Lemma 14 (pruneF_tree_to_stack).

$$\forall\ t\ t'\ b,\ \text{valid\_tree}\ t \to \text{failed}\ t \to \text{prune}\ b\ t = \text{Some}\ t' \to \\ \forall\ l\ \sigma,\ \text{tree\_to\_stack}\ t\ \sigma\ l = \text{tree\_to\_stack}\ t'\ \sigma\ l$$

Proof. By induction on the tree t.

The first direction of the equivalence states that the execution of a valid tree is matched by the execution of the stack obtained by translating the tree. Moreover, the unexplored alternatives returned as a tree correspond to those returned as a stack.

Theorem 2 (runT_to_runS).

$$\forall\ p\ t\ \sigma\ r,\ \text{let}\ v := \text{vars}\ t\ \cup\ \text{vars}\ \sigma\ \text{in valid\_tree}\ t \to \text{runT}'\ p\ v\ \sigma\ t\ r \to \\ \begin{cases} \text{runS}\ p\ v\ (\text{tree\_to\_stack}\ t\ \sigma\ [])\ \text{None} & \text{if } r = \text{Zero} \\ \text{runS}\ p\ v\ (\text{tree\_to\_stack}\ t\ \sigma\ [])\ (\text{Some}\ (\sigma', [])) & \text{if } r = \text{One}\ \sigma' \\ \text{runS}\ p\ v\ (\text{tree\_to\_stack}\ t\ \sigma\ [])\ (\text{Some}\ (\sigma',\ \text{tree\_to\_stack}\ t'\ \sigma\ [])) & \text{if } r = \text{Many}\ \sigma'\ t' \end{cases}$$

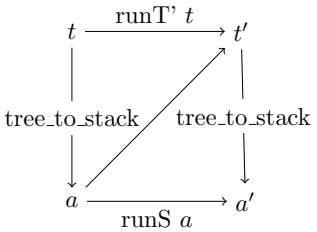
Proof. We proceed by induction on the execution of runT'. There are five cases to consider. The first (similar) cases arises from StopOT and StopMT; they deal with successful trees. A successful tree corresponds to a stack that start with an empty list, in both cases we conclude using StopS. The case for StepT requires treating two subcases: step evaluates either a cut or a call. Accordingly, we apply CutS or CallS, followed by the induction hypothesis. The case for BackT is silent: it represents the non-injective behavior of tree_to_stack; however, thanks to lemma 14 and the induction hypothesis, the conclusion is proved. The last case arises from the derivation FailT. It is easily proved using FailS and the fact that if prune returns None when trying to erase failure in t, then tree_to_stack applied to t returns the empty list.

The converse direction is stated following [2]: instead of using a function to translate a stack into a tree it states that any tree that translates to the given stack has an equivalent execution, i.e. gives the same solution and returns an unexplored tree that translates to the unexplored stack.

Theorem 3 (runS_to_runT).

$$\forall\ p\ v\ a\ r,\ \text{runS}\ p\ v\ a\ r \to \forall\ \sigma\ t,\ \text{valid\_tree}\ t \to \text{tree\_to\_stack}\ t\ \sigma\ [] = a \to$$
$$\begin{cases} \text{runT}'\ p\ v\ \sigma\ t\ \text{Zero} & \text{if } r = \text{None} \\ \text{runT}'\ p\ v\ \sigma\ t\ (\text{One}\ \sigma') & \text{if } r = \text{Some}\ (\sigma',\ []) \\ \exists\ t',\ \text{runT}'\ p\ v\ \sigma\ t\ (\text{Many}\ \sigma'\ t') \wedge \text{tree\_to\_stack}\ t'\ \sigma\ [] = a' & \text{if } r = \text{Some}\ (\sigma',\ a') \end{cases}$$

Proof. The proof is based on the idea explained in [2, Section 3.5.1] and depicted in the figure on the right: we have a tree t whose translation is a, we proceed by induction on the execution of runS. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to a stack is a'.



Stating the converse direction of theorem 2 in a symmetric way would require a function stack_to_tree transforming a stack a into a tree t. Writing this function is far from trivial, since the structure of the tree is lost by $\mathcal{B}_S$ and runS that intuitively keep the state as a big disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover, one would need to define a notion of valid stack that is equally intricate.

The constructive nature of the logic of Rocq tells us that the proof above provides an effective function translating stacks into trees. However, that function also uses an execution trace of the stack-based semantics, from which it recovers the missing tree structure.

6 Case study: determinacy analysis

As an application of the tree semantics, we mechanize the soundness proof of the determinacy-checking algorithm introduced in version 3 of the Elpi language [6].

The static checker takes as input the signatures of the predicates declared by the user and verifies that the rules of a deterministic predicate generate at most one solution; we call this kind of predicate a function. Thanks to this static verification, the user can better control unexpected backtracking.

A predicate behaves deterministically if two conditions are satisfied: *mutual exclusion* guarantees that at most one rule can be successfully applied to any given query, whereas *rule body checking* ensures that each rule does not create choice points, for example by calling non-deterministic predicates.

Determinacy checking is strongly related to the modes of predicates: we differentiate between input and output arguments. A deterministic call relates to its inputs only one output. In the literature, we find several works on determinacy checking, namely [18,11,4]. These works need a mode checker that statically ensures that, in a query with ground inputs, each recursive call is performed with ground inputs.

Our mechanization implements the unification algorithm by Martelli and Montanari [13], which we prove terminating using the standard lexicographic ordering, correct and complete.

In the following definitions and lemmas, we use the notation $x \cap y = \varnothing$ to denote that the two sets x and y are disjoint. When the arguments are terms, as in $t_1 \cap t_2 = \varnothing$, we mean that the two terms do not share any variables.

The first notion to provide is the application of an acyclic substitution:

Definition 2 (deref).

$$\begin{aligned}
\sigma\ (\text{Symb}\ n) &= \text{Symb}\ n \\
\sigma\ (\text{Pred}\ n) &= \text{Pred}\ n \\
\sigma\ (\text{Var}\ n) &= t \quad \text{if}\ (n, t) \in \sigma \\
\sigma\ (\text{Var}\ n) &= \text{Var}\ n \quad \text{otherwise} \\
\sigma\ (\text{App}\ t_1\ t_2) &= \text{App}\ (\sigma\ t_1)\ (\sigma\ t_2)
\end{aligned}$$

Definition 3 (acyclic). $\lambda\sigma.\text{dom}\ \sigma \cap \text{codom}\ \sigma = \varnothing$

Note that this definition of acyclicity entails that the substitution is necessarily self applied, i.e. $\sigma \coloneqq \{X \mapsto f\ Y, Y \mapsto a\}$ is not acyclic, whereas the equivalent $\sigma' \coloneqq \{X \mapsto f\ a\}$ is. The algorithm by Martelli and Montanari generates substitutions that are acyclic in this sense.

Lemma 15 (unify_correct).

$$\forall\ t_1\ t_2\ s\ s',\ \text{acyclic\_sigma}\ s \to \text{unify}\ t_1\ t_2\ s = \text{Some}\ s' \to s'\ t_1 = s'\ t_2$$

Lemma 16 (unify_complete). $\forall\ t_1\ t_2\ s,\ \text{acyclic\_sigma}\ s \to$

$$(\exists\ s',\ \text{acyclic\_sigma}\ s' \wedge s'\ (s\ t_1) = s'\ (s\ t_2)) \to \exists\ s'',\ \text{unify}\ t_1\ t_2\ s = \text{Some}\ s''$$

The Elpi language adopts a restricted unification routine for input arguments, called matching [1,21]. Similarly to unify, matching takes the query term

t_1 and the pattern t_2 and returns an optional substitution σ' that does not assign any variable in the query arguments that are in input position.

To address this need, we extend the Martelli-Montanari unification procedure so that it takes a set of frozen variables, i.e. variables that cannot be assigned. The function matching has the following property:

Lemma 17 (matching_disj).

$$\forall t_1\ t_2\ \sigma\ \sigma', \text{acyclic}\ \sigma \rightarrow \text{matching}\ t_1\ t_2\ \sigma = \text{Some}\ \sigma' \rightarrow \sigma'\ t_1 = \sigma\ t_2$$

In particular, we can see how the returned substitution σ', differently from lemma 15, does not affect variables in $\sigma\ t_2$.

Corollary 1 (matching_same).

$$\forall t\ q\ \sigma\ \sigma', \text{acyclic}\ \sigma \rightarrow \text{matching}\ t\ q\ \sigma = \text{Some}\ \sigma' \rightarrow \sigma'\ q = \sigma\ q$$

The backchain function uses matching or unify on the arguments q depending on whether the argument is in *input* or *output* mode.

For the static checker, and in particular in order to validate rule mutual exclusion, we need to ensure that no two rules can be used on the same query in the absence of cut. We exploit the behavior of the matching over input arguments to guarantee this property: mutual exclusion ensures that every pair of rules for a deterministic predicate has an input term that does not unify.

An important property we derive from the matching and unification procedures is the following. Given three pairwise variable-disjoint terms h_1, h_2, and q, representing two rule heads in the database and a query, if both heads match the query q, then the two heads must unify. This idea is formalized by the following lemma.

Lemma 18 (matching_unify_transP).

$$\lambda\ h_1\ h_2\ q.\ h_1 \cap h_2 = \varnothing \rightarrow h_1 \cap q = \varnothing \rightarrow h_2 \cap q = \varnothing \rightarrow \\ \text{matching}\ h_1\ q\ \varnothing \rightarrow \text{matching}\ h_2\ q\ \varnothing \rightarrow \text{unify}\ h_1\ h_2\ \varnothing$$

Proof. By lemma 17, we now that the matching of h_1 (resp. h_2) and q gives a substitution σ_1 (resp. σ_2) where the variables in q are not assigned, i.e. its domain only contains variables in h_1 (resp h_2). Using lemma 16, on the conclusion, we need to prove that it exists a substitution σ' such that $\sigma'\ h_1 = \sigma'\ h_2$. This substitution is the composition of σ_1 and σ_2, that is $\sigma' = \lambda x.\sigma_1(\sigma_2\ x)$. Since h_1 and h_2 are disjoint, also are the domain of σ_1 and σ_2, therefore $\sigma' h_1 = \sigma_2(\sigma_1\ h_1)$ and $\sigma' h_2 = \sigma_2(\sigma_1\ h_2)$. We can know conclude the proof: $\sigma'\ h_1 = \sigma_2(\sigma_1\ h_1) = \sigma_2\ q = q = \sigma_2\ h_2 = \sigma' h_2$.

Finally, we also need a lemma certifying that the refreshing variables operation performed at runtime during backchaining does not affect unification. In particular, a refreshing renaming f for a term t is a function from variables to variables such that: (i) f is defined on all variables in t, i.e. every variable is renamed; and (ii) f is injective, i.e. it does not identify distinct variables. Refreshing therefore produces α-equivalent terms.

Definition 4 ($=_\alpha$). $\lambda t_1\ t_2.\exists(r : \text{renaming\_for}\ t_2), t_1 = \text{rename}\ r\ t_2$

Lemma 19 (unif_ren_ac). $\forall\ t_1\ t_2\ t_1'\ t_2',$

$$t_1 =_\alpha t_1' \wedge t_2 =_\alpha t_2' \wedge t_1 \cap t_2 = \varnothing \wedge t_1' \cap t_2' = \varnothing \rightarrow \\ \text{unify}\ t_1\ t_2\ \varepsilon \rightarrow \text{unify}\ t_1'\ t_2'\ \varepsilon$$

In fig. 1, the rules of the program are equipped with three signatures, one per predicate. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, their types, and whether the predicate is deterministic. They indicate that `f` and `g` are expected to be functions, as specified by the `func` keyword, whereas `r` is a relation, as indicated by the `pred` keyword. The `->` symbol separates inputs from outputs; therefore, all three predicates are expected to take an `int` as input and return an `int` as output.

The signatures of the program are stored in the `sig` field of the program and are encoded as described in [6]: we distinguish data from predicates, and predicates are classified as deterministic or non-deterministic.

A program execution behaves deterministically if, given a program, a substitution, and an input tree for the runT inductive, the execution either fails or succeeds with no alternatives left to explore.

Definition 5 (is_det).

$$\lambda\ p\ \sigma\ v\ t.\ \forall\ r,\ \text{runT}'\ p\ v\ \sigma\ t\ r \rightarrow r = \text{Zero} \vee \exists\ \sigma,\ r = (\text{One}\ \sigma)$$

Definition 6 (tm_is_det). *Given a program $\mathbb{P}$ and a term t, we say that t is deterministic if its head is a predicate p which is declared as deterministic in $\mathbb{P}$.*

The theorem we want to prove is the following one:

Theorem 4 (det_check_call).

$$\forall\ p\ \sigma\ t\ v,\ \text{check\_program}\ p \rightarrow \text{tm\_is\_det}\ p\ t \rightarrow \text{is\_det}\ p\ \sigma\ v\ (\text{Unexplored}\ (\text{call}\ t))$$

Here, *check_program* denotes the function that checks the program with respect to mutual exclusion and rule checking.

The proof of this lemma proceeds by induction on the program execution. However, without further work, the induction hypothesis is too weak since it talks about Unexplored trees (i.e. atoms), whereas we need it on any tree.

To overcome this issue, we introduce the notion of deterministic trees, written *det_tree*. Intuitively, a deterministic tree is a tree whose interpretation does not generate choice points. The definition is shown in fig. 9.

For a call to a term t, the tree is deterministic whenever t calls a deterministic predicate. The recursive cases correspond to the And and Or nodes.

A conjunction behaves deterministically when its right-hand side is a deterministic tree. The test prune(success A) A == None checks whether the left-hand side may still contain internal choice points due to Or nodes. Such choice

```
Fixpoint det_tree (p: program) A :=
  match A with
  | Unexplored cut => true
  | Unexplored (call t) => tm_is_det p t
  | KO | OK => true
  | And A B0 B =>
      det_tree p B &&
      if prune (success A) A == None
      then det_tree p A || has_cut B
      else
        (det_tree p A || (has_cut B && has_cut_seq B0)) &&
        det_tree_seq p B0
  | Or None _ B => det_tree p B
  | Or (Some A) _ B =>
      det_tree p A &&
      if has_cut A then det_tree p B
      else (B == KO)
  end.
```

Fig. 9: The *det_tree* predicate

points could later be activated by backtracking and therefore interact with the reset point.

If the test succeeds, then either A is deterministic, or B contains a superficial cut. In the latter case, the cut commits to the first solution produced by A, preventing evaluating the potential alternatives generated by the interpretation of A.

Otherwise, the reset point must itself behave deterministically, meaning that all calls inside it are deterministic. Moreover, as in the previous case, either A is deterministic, or both B and the reset point contain a cut that prevents the non-deterministic behaviour of the evaluation of A.

For Or trees, the interesting case is when the left-hand side contains an tree. In this case, A must be deterministic. If A contains a superficial cut, then B must also be deterministic. Indeed, if A succeeds, the cut discards B; otherwise, if A fails before reaching the cut, then no solution is produced by A and B is evaluated.

Finally, if A does not contain a cut, then B must be equal to KO, that is, a tree already discarded by a cut. This condition is essential. Otherwise, if A succeeds without a superficial cut and B is not KO, then B could still produce a successful branch, leading to non-deterministic behaviour.

The following lemmas show that deterministic trees are preserved by the operations step and prune.

Lemma 20 (det_tree_step).

$$\forall\ \text{pr}\ v\ \sigma_1\ A\ r,\ \text{check\_program pr} \to \text{det\_tree pr}\ A \to \text{step}\ u\ \text{pr}\ v\ \sigma_1\ A = r \to \text{det\_tree pr}\ r.2$$

Lemma 21 (det_tree_prune).

$$\forall\ p\ A\ B\ b,\ \text{det\_tree}\ p\ A \to \text{prune}\ b\ A = \text{Some}\ B \to \text{det\_tree}\ p\ B$$

Finally, we can prove the following theorem:

Lemma 22 (det_check_tree).

$$\forall\ \sigma\ v\ p\ t,\ \text{check\_program}\ p \to \text{det\_tree}\ p\ t \to \text{is\_det}\ p\ \sigma\ v\ t$$

Our target theorem det_check_call is a corollary of det_check_tree.

7 Related work

The only mechanization of Prolog's semantics that we could find is Pusch's Isabelle/HOL formalization [17], which is based on the model of Debray and Mishra [3, Sec. 4.3]. That work starts from this semantics and refines it by introducing optimizations commonly found in the Warren abstract machine [20,1]. Unlike our case study, it does not use the semantics to prove properties of program execution; consequently, it does not encounter the difficulty described in section 4, which led us to develop a semantics in which the effect of cut is more explicit. In this sense, the two approaches are complementary: taken together, they suggest that the tree-based semantics is equivalent to an optimized stack-based one.

Another relevant work is due to Kriener and King [9], but we could not find their Rocq source code. Their semantics is denotational and cannot easily account for all the features covered in [6], although it would have been sufficient for the first-order fragment considered in this paper.

The proof technique we use to relate the two semantics is inspired by [2], where Bertot mechanizes the correctness of a compiler. He relates the semantics of an imperative language to that of its compiled assembly code and proves their equivalence. His approach uses the compiler as the translation function from one language to the other, but does not introduce a decompiler, just as we do not define a function from stacks to trees. To show that the assembly code behaves like the imperative language, he proceeds by induction on the execution of the assembly program. We adopt the same strategy in the proof of theorem 3.

For the static analysis of determinacy, we use a semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the Elpi interpreter, which is in turn inspired by B-Prolog's "matching-clauses" [1,21].

Applying our results to a semantics based entirely on unification is possible by adding two extra assumptions to theorem 4. First, the initial query must

have ground inputs. Second, the program must be well-moded. Well-moded predicates produce ground outputs when given ground inputs; hence, starting from an initial query with ground inputs, well-moded programs behave exactly as if input arguments were matched. In other words, lemma lemma 17 could be reformulated using unification, at the cost of an additional assumption that the input arguments are ground; under that assumption, unification and matching coincide.

8 Conclusion

This paper describes two operational semantics for Prolog with cut. The first semantics is based on And–Or trees: it is well suited both to graphical illustrations of the scope of cut and to proving lemmas about the effect of cut evaluation when reasoning about disjunctions. The second semantics is based on a stack of alternatives: it is closer to actual Prolog implementations, including the first-order fragment of Elpi, but it loses the explicit structure of the tree-based semantics and is therefore less convenient for reasoning about cut. We show how to translate trees into stacks and prove the two semantics equivalent. Using the tree-based semantics, we mechanize a soundness proof of the first-order fragment of the determinacy checker of [6], namely that a call to a deterministic predicate returns at most one solution.

The mechanization consists of approximately 2,500 lines of specifications and about 5,000 lines of ssreflect proofs based on the Mathematical Components library [12] and its finmap [14] extension. Roughly 30% of the code is dedicated to the tree semantics, in particular the proofs in section 3.4, 15% to the stack semantics, and 35% to the equivalence proof. The remaining 20% is devoted to the determinacy checker. The mechanization took us approximately 8 months. We believe that the ssreflect proof language, and the proof style it advocates, helped us carry out the many refactorings that the mechanization underwent. The finmap library provides comprehensive support for reasoning about finite maps and finite sets over a countable domain. In particular, it helped us express conditions such as the disjointness between sets of variables and the domain of a substitution, as well as define the union of disjoint substitutions.

To obtain an axiom-free mechanization, we had to provide an implementation of the first-order unification algorithm, since we could not find an existing Rocq mechanization. Describing this development in detail is out of scope for this paper. It amounts to about 1,700 lines of ssreflect code, including 200 lines for the termination proof based on a lexicographic ordering and 800 lines for correctness and completeness. Because of the specific form of lemmas 17 and 19, which deals with matching and unification of terms with disjoint support, we did not need to prove that the algorithm produces most-general unifiers.

To fully model the Elpi language and mechanize [6] entirely, we would need to account for higher-order variables, that is, variables that represent predicates. This extension is ongoing and largely orthogonal to the present paper, since cut and higher-order variables do not interact in subtle ways. The main difficulty

is that substitutions may contain predicates, and these predicates must have signatures compatible with those assumed statically, in the sense of the $\subseteq$ relation defined in [6, Section 5]. Dynamic predicates also require minor changes to the conditions for mutual exclusion.

Acknowledgments. TODO: hidden for review

References

1. Aït-Kaci, H.: Warren's Abstract Machine: A Tutorial Reconstruction. The MIT Press (08 1991). https://doi.org/10.7551/mitpress/7160.001.0001, https://doi.org/10.7551/mitpress/7160.001.0001
2. Bertot, Y.: A certified compiler for an imperative language. Tech. Rep. RR-3488, INRIA (September 1998), https://inria.hal.science/inria-00073199v1
3. Debray, S.K., Mishra, P.: Denotational and operational semantics for prolog. J. Log. Program. **5**(1), 61–91 (Mar 1988). https://doi.org/Debray1988, https://doi.org/Debray1988
4. Debray, S.K., Warren, D.S.: Functional computations in logic programs. vol. 11, p. 451–481. Association for Computing Machinery (Jul 1989). https://doi.org/10.1145/65979.65984, https://doi.org/10.1145/65979.65984
5. Dunchev, C., Guidi, F., Coen, C.S., Tassi, E.: ELPI: fast, embeddable, λProlog interpreter. In: Proceedings of LPAR. LNCS, vol. 9450, pp. 460–468. Springer (2015). https://doi.org/10.1007/978-3-662-48899-7_32, https://inria.hal.science/hal-01176856v1
6. Fissore, D., Tassi, E.: Determinacy checking for Elpi: an higher-order logic programming language with cut. In: Amin, N., Arias, J. (eds.) Practical Aspects of Declarative Languages. pp. 77–95. Springer Nature Switzerland, Cham (2026)
7. Hanus, M., Prott, K.O.: Determinism Types for Functional Logic Programming. In: Proceedings of PPDP. pp. 47–59 (2025)
8. Henderson, F., Somogyi, Z., Conway, T.: Determinism analysis in the mercury compiler. In: Proceedings of Australian Computer Science Conference. pp. 337–346 (08 1996)
9. Kriener, J., King, A.: Redalert: Determinacy inference for prolog. vol. 11, pp. 537–553. Cambridge University Press (2011)
10. López-García, P., Bueno, F., Hermenegildo, M.: Determinacy analysis for logic programs using mode and type information. In: Etalle, S. (ed.) Logic Based Program Synthesis and Transformation. pp. 19–35. Springer Berlin Heidelberg, Berlin, Heidelberg (2005)
11. Lu, L., King, A.: Determinacy inference for logic programs. vol. 3444, pp. 108–123 (04 2005). https://doi.org/10.1007/978-3-540-31987-0_9
12. Mahboubi, A., Tassi, E.: Mathematical Components. Zenodo (Sep 2022). https://doi.org/10.5281/zenodo.7118596, https://doi.org/10.5281/zenodo.7118596
13. Martelli, A., Montanari, U.: An efficient unification algorithm. ACM Trans. Program. Lang. Syst. **4**(2), 258–282 (Apr 1982). https://doi.org/10.1145/357162.357169, https://doi.org/10.1145/357162.357169
14. math-comp: finmap — finite maps for mathcomp (2026), https://github.com/math-comp/finmap
15. Miller, D.: A logic programming language with lambda-abstraction, function variables, and simple unification. In: Extensions of Logic Programming. pp. 253–281. Springer (1991)

16. Miller, D., Nadathur, G.: Programming with Higher-Order Logic. Cambridge University Press (2012)
17. Pusch, C.: Verification of compiler correctness for the wam. In: Goos, G., Hartmanis, J., van Leeuwen, J., von Wright, J., Grundy, J., Harrison, J. (eds.) Theorem Proving in Higher Order Logics. pp. 347–361. Springer Berlin Heidelberg, Berlin, Heidelberg (1996)
18. Somogyi, Z., Henderson, F., Conway, T.: The execution algorithm of mercury, an efficient purely declarative logic programming language. vol. 29, pp. 17–64 (1996). https://doi.org/https://doi.org/10.1016/S0743-1066(96)00068-4, https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684, high-Performance Implementations of Logic Programming Systems
19. SWI-Prolog Documentation: Predicate !/0 — cut (built-in predicate) (2026), https://www.swi-prolog.org/pldoc/doc_for?object=!/0, accessed via SWI-Prolog PlDoc reference
20. Warren, D.H.: An Abstract Prolog Instruction Set. Tech. Rep. Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA (October 1983), https://www.sri.com/wp-content/uploads/2021/12/641.pdf
21. Zhou, N.f.: The language features and architecture of b-prolog. Theory Pract. Log. Program. **12**(1–2), 189–218 (Jan 2012). https://doi.org/10.1017/S1471068411000445, https://doi.org/10.1017/S1471068411000445